

Approach — Conversational SHL Assessment Recommender

A stateless FastAPI service that turns a vague hiring intent into a grounded shortlist of SHL assessments through dialogue. It **clarifies**, **recommends**, **refines**, and **compares**, and **refuses** off-topic/legal/injection input — never recommending anything outside the scraped catalog.

Design choices (and why)

- **One LLM call per turn.** No multi-hop agent loop. Keeps every /chat inside the 30 s budget and keeps the system defensible. The turn flow is: guard → build_query → retrieve → inject_anchors → llm_turn → hydrate → validate.
- **The LLM selects by entity_id; code hydrates the fields.** The model is given a candidate list and returns only chosen ids. Code looks each id up in the catalog and builds {name, url, test_type}. **Hallucinated names/URLs are structurally impossible** — the model never authors a URL, and any id not in the catalog is dropped. test_type is derived in code from the catalog keys[] (8-way map), never by the model.
- **Stateless re-derivation.** No per-conversation state. Every turn re-derives the shortlist from the full message history. The one subtlety: the API only echoes back our reply text, not the structured recs — so the policy restates the shortlist by name each turn, and we **reconstruct the committed shortlist by matching catalog names in the last assistant reply**, feed it to the model as "current shortlist", keep those items in the candidate pool, and use it as the fallback. This is what makes *refine* ("drop REST, add Docker") and confirmation turns work statelessly.
- **Never 500, never fake success.** Every failure path (bad JSON, LLM timeout, rate limit, empty history, malformed body) returns a valid response with recommendations: [] and an honest clarify reply.

Stack (justification)

FastAPI + Pydantic (schema is hard-scored, so typed models enforce it); **BM25 (rank_bm25)** for retrieval — the catalog is 377 items, so a heavy vector DB is unwarranted and BM25 measurably wins (below); **Gemini 2.5 Flash** via its OpenAI-compatible endpoint (JSON mode) — free, fast, and its 15 req/min · 1M tokens/min budget comfortably survives the evaluator's many-trace replay. (We first tried Groq but its free tier throttled us on tokens-per-minute *and* tokens-per-day; the provider is a config swap.) All tunables live in config.py/.env.

Data

Downloaded once, normalized to data/catalog_normalized.json (committed, so boot needs no network). **377 items**, all with links/ids/descriptions (verified). The source has unescaped control chars in descriptions → parsed leniently, re-emitted as strict JSON. keys[] → test_type mapping: A/B/C/D/E/K/P/S; the catalog is heavily Knowledge & Skills (240/377).

Retrieval — what we measured

Metric = **candidate-pool recall**: fraction of each trace's gold items that reach the pool sent to the LLM (the ceiling on final Recall@10). Measured offline on the 10 traces, so it needs no LLM:

Retrieval variant	Mean pool recall
BM25, single concatenated query	0.778
BM25, per-turn multi-query + RRF fusion + name/keys field-boost	0.832
Dense only (bge-small, fastembed)	0.764
Hybrid BM25+dense (RRF)	0.832 (no gain)

What didn't work: (1) a single concatenated query buries single-skill tests (e.g. C9's SQL, Docker) behind broadly-matching items — fixed by running BM25 per user-turn and fusing with RRF (surfaces each skill from the turn that names it). (2) **Dense retrieval did not help** — dense-only was worse and hybrid tied BM25, because the residual gaps are expert-bundled companion items (reports, battery extras) that embeddings don't bridge either. So we **kept the lighter BM25-only stack** (no 130 MB model in the deploy). We also inject 4 config anchors (OPQ32r, Verify G+, Graduate Scenarios, DSI) into the pool — a retrieval boost the policy may layer and must drop on request.

Prompt / policy design

A single system prompt encodes the behavior contract: the **clarify gate** (vague opener → ask one question; role + concrete anchor → recommend, even on turn 1), recommend/refine (tight lists, droppable default layering, honesty about catalog gaps), compare (grounded in candidate descriptions), and refuse (off-topic, legal → point to counsel, injection). Compressed few-shots (one per behavior, from the traces) anchor the JSON shape. A deterministic regex guard() refuses obvious injection before the LLM call. A **clarify-loop cap** forces a commit after 2 assistant turns so we never blow the 8-turn budget.

Evaluation

`eval/run_eval.py` replays each trace's user turns against `/chat`, tracks the final shortlist, and computes **Recall@10 + behavior probes** (schema valid on every response, all URLs in-catalog, ≤10 items, vague→no-recs, injection→refuse, off-topic→refuse, legal→refuse, edits honored for C9 drop REST / C10 drop OPQ).

Results: mean Recall@10 ≈ 0.5 on public traces (Groq 8b-instant); probes 9/9. Biggest bug found and fixed via this harness: early runs returned the *top-3 fallback candidates* on confirmation turns, overwriting good shortlists (mean 0.22). Root cause was two-fold — LLM **rate-limiting** (429s cascading to fallback; fixed with `retry-after` backoff, smaller prompts, and finally moving to Gemini's higher free-tier limits) and **missing stateless carry-forward** (fixed as above), after which the shortlist correctly persists/mutates across all of C9's 7 turns.

AI tools

Built with **Claude Code** (Anthropic) for implementation, debugging, and the eval harness; all design decisions above are ones we can defend.